UniSENAI SANTA CATARINA
Análise e Desenvolvimento de Sistemas

Introdução a Banco de Dados

Primeiros passos com MySQL

Material de apoio da aula de 4 horas

Prof. Me. Janderson Barbosa da Silva

Joinville, Santa Catarina

Como usar este material

Este é o material da sua primeira aula de banco de dados. Ele não supõe que você já saiba alguma coisa do assunto: começa do zero e vai devagar.

Você pode usá-lo de três formas. Antes da aula, ler os capítulos 1 e 2 ajuda a chegar com o vocabulário na cabeça. Durante a aula, ele evita que você precise copiar os comandos do slide: está tudo aqui, escrito e comentado. Depois da aula, os capítulos 3 e 4 funcionam como manual de consulta enquanto você pratica.

A última página é uma colinha com todos os comandos da aula em um lugar só. Se você for imprimir só uma página, imprima aquela.

Como ler os blocos de código. Os blocos com fundo cinza e letra de máquina de escrever são comandos para digitar no computador. Digite exatamente como está, incluindo o ponto e vírgula do final.

O que você vai saber fazer ao final

- Explicar o que é um banco de dados e por que ele é melhor do que um caderno ou uma planilha.
- Reconhecer as partes de uma tabela: linha, coluna e tipo de dado.
- Criar um banco de dados e uma tabela usando comandos SQL.
- Guardar informações na tabela e depois encontrá-las com filtros.
- Corrigir e apagar informações com segurança.
- Ler uma mensagem de erro do MySQL e entender o que ela quer dizer.

Uma coisa importante antes de começar. Errar um comando de SQL não quebra nada e não apaga nada. O banco de dados simplesmente recusa o comando e explica o motivo. Pode errar à vontade — é assim que se aprende.

1. O que é um banco de dados

1.1 Você já usa banco de dados todo dia

Antes de qualquer definição, vale perceber que você convive com bancos de dados o tempo inteiro, mesmo sem saber.

- Você abre o Instagram e suas fotos aparecem. Alguém guardou, em algum lugar, que aquelas fotos são suas.
- Você abre o aplicativo do banco e o saldo aparece. Alguém guardou quanto você tem.
- O professor lança sua nota e, na semana seguinte, ela continua lá no portal do aluno.
- Você busca um produto em um site e ele encontra, no meio de milhões, em menos de um segundo.

A regra é simples: toda vez que uma tela mostra uma informação que alguém guardou antes, existe um banco de dados por trás.

1.2 Como as pessoas guardavam informação

Guardar informação não é um problema novo. O que mudou foi a ferramenta.

- **No papel.** Caderno, agenda, ficha, caderneta de telefone. Simples e barato.
- **Na planilha.** Excel, Google Planilhas. Já dá para ordenar, somar e copiar.
- **No banco de dados.** Vários usuários ao mesmo tempo, milhões de linhas, com regras e segurança.

Nenhum dos três é errado. O que muda é o tamanho do problema que cada um aguenta. Uma lista de dez compras cabe num papel. Os clientes de uma loja, não.

1.3 A agenda de papel: o que dá errado

Imagine uma agenda de papel com quinhentos contatos. O que aconteceria?

- **Achar demora.** Para encontrar o telefone do João, você folheia página por página.
- **Repete.** O mesmo contato acaba anotado duas vezes, em páginas diferentes, com números diferentes.
- **Vira bagunça.** Cada um anota de um jeito: “47 9999-1111”, “4799991111”, “(47) 99999-1111”.
- **Uma pessoa por vez.** Se você está com a agenda na mão, mais ninguém consegue usar.
- **Rasura e some.** Apagou com borracha, ficou ilegível. Perdeu a agenda, perdeu tudo.
- **Ninguém confere.** Nada impede você de escrever uma data que não existe, como 30 de fevereiro.

Guarde essa lista. Cada um desses problemas é resolvido por alguma coisa que você vai aprender nesta aula.

1.4 Então: o que é um banco de dados?

> **A definição.** Um banco de dados é um conjunto de informações guardadas de forma organizada, para que o computador consiga achar, mostrar e mudar essas informações rapidamente.

Três palavras dessa frase merecem atenção.

- **Guardadas.** A informação continua lá depois que você fecha o programa e desliga o computador.
- **Organizadas.** Tudo tem um lugar certo e um formato certo. Não é um monte de texto solto.
- **Rapidamente.** Encontrar um contato entre um milhão leva uma fração de segundo.

É a palavra “organizadas” que faz a diferença. Um arquivo de texto também guarda informação — mas sem organização, sem regras e sem jeito rápido de procurar.

1.5 Quem cuida do banco de dados: o SGBD

O banco de dados não se cuida sozinho. Existe um programa responsável por ele, chamado SGBD, que é a sigla de Sistema Gerenciador de Banco de Dados.

- **Ele guarda.** Grava a informação no disco e garante que ela continue lá amanhã.
- **Ele protege.** Controla quem pode ver e quem pode alterar cada coisa.
- **Ele entrega.** Quando alguém pede, encontra a informação e devolve.
- **Ele organiza a fila.** Se dez pessoas pedirem ao mesmo tempo, atende todas sem bagunçar.

Existem vários SGBDs no mercado: MySQL, MariaDB, PostgreSQL, SQL Server e Oracle são os mais conhecidos. Nesta aula vamos usar o MySQL.

1.6 O MySQL

O MySQL é o SGBD que vamos usar. Ele é gratuito, você pode instalar hoje mesmo no seu computador, e está por trás de uma parte enorme dos sites que existem na internet.

Ele já vem pronto dentro do XAMPP, que é o pacote que vamos instalar no capítulo 3. E o que você aprender aqui continua valendo depois: os comandos são quase os mesmos em outros bancos, como PostgreSQL e SQL Server.

> **Se o seu computador tiver MariaDB em vez de MySQL** não tem problema: o MariaDB é praticamente a mesma coisa, e tudo o que está neste material funciona igual.

1.7 Três nomes que confundem todo mundo

Esse é o ponto em que quase todo iniciante se embaralha. Vale ler com calma.

Termo	O que é	No nosso caso
Banco de dados	A informação guardada	A lista de contatos, com nome, telefone e e-mail de cada um
SGBD	O programa que cuida dessa informação	O MySQL, ligado no computador esperando pedidos
SQL	A linguagem para pedir coisas ao programa	Os comandos que vamos digitar

Uma comparação ajuda a fixar: na biblioteca, o banco de dados é o acervo de livros; o SGBD é o bibliotecário; e o SQL é o idioma em que você faz o pedido a ele.

Erro comum. Dizer que “MySQL é um banco de dados”. Não é. MySQL é o programa que cuida do banco de dados. O banco de dados é a informação.

1.8 O que é SQL

SQL é a linguagem que o banco de dados entende. Você escreve uma frase e o banco responde. E o melhor: SQL é quase inglês, então você não precisa decorar — precisa entender o que cada palavra está pedindo.

Você escreve	E o banco entende
`SELECT * FROM contatos`	“Mostre tudo o que está na lista de contatos.”
`SELECT nome FROM contatos`	“Mostre só o nome de cada contato.”
`INSERT INTO contatos ...`	“Guarde este contato novo na lista.”
`UPDATE contatos SET ...`	“Corrija esta informação da lista.”
`DELETE FROM contatos ...`	“Apague este contato da lista.”

Nesta aula você vai aprender exatamente esses comandos — e mais alguns poucos para criar o banco e a tabela. É pouca coisa, e já dá para fazer muita coisa.

2. A tabela

2.1 A tabela é uma lista organizada

Dentro de um banco de dados, a informação fica guardada em tabelas. Uma tabela é uma lista com cabeçalho, igual a uma folha de caderno bem organizada.

id	nome	telefone	email
1	Maria Silva	47999991111	maria@email.com
2	João Pereira	47999982222	joao@email.com
3	Ana Souza	47999973333	ana@email.com

Uma tabela guarda um tipo de coisa. Esta guarda contatos. Se depois quisermos guardar produtos também, criamos outra tabela.

2.2 Coluna, linha e campo

- **Coluna** é a característica que você quer guardar de todo mundo. Na tabela acima existem quatro colunas: id, nome, telefone e email. Toda linha tem exatamente as mesmas colunas.
- **Linha** é um item completo da lista — um contato inteiro. A segunda linha é o João Pereira, com o telefone e o e-mail dele. Também se diz “registro”.
- **Campo** é o encontro de uma linha com uma coluna: uma casinha da tabela. O campo “telefone do João” contém 47999982222.

> **Vocabulário.** Linha também é chamada de registro. Coluna também é chamada de campo ou de atributo. Todos esses nomes aparecem em livros e em provas — não estranhe.

2.3 Toda coluna tem um tipo

Aqui está a primeira grande diferença entre uma tabela do banco e uma aba de planilha.

Na planilha	Na tabela do banco
A célula aceita qualquer coisa	Você declara antes o que cada coluna aceita
Na coluna “idade” cabe 30, “trinta” ou “não sei”	Se a coluna é de números, a palavra “trinta” não entra
Ninguém reclama na hora	O banco recusa na hora e avisa
O problema aparece depois, na hora de somar	O dado errado nunca chega a ser guardado

Escolher o tipo certo de cada coluna é uma das decisões mais importantes ao criar uma tabela. É por isso que a próxima seção é inteirinha sobre isso.

2.4 Os cinco tipos que você precisa hoje

O MySQL tem dezenas de tipos. Cinco deles resolvem a grande maioria dos casos, e são estes que você precisa conhecer agora.

Tipo	Guarda	Use para	Exemplo de valor
`INT`	Números inteiros, sem vírgula	idade, quantidade, id	42
`VARCHAR(100)`	Texto de até 100 caracteres	nome, e-mail, telefone	'Maria Silva'
`DATE`	Uma data	data de nascimento	'2005-03-17'
`DATETIME`	Data e hora	momento do cadastro	'2026-08-25 14:30:00'
`DECIMAL(10,2)`	Número com casas decimais	preço, salário	1499.90

O número entre parênteses do VARCHAR é o tamanho máximo. VARCHAR(100) aceita até cem caracteres: um nome cabe; um texto longo, não. Já o DECIMAL(10,2) quer dizer “dez dígitos no total, sendo dois depois da vírgula”.

2.5 Cuidado: telefone é texto

Este é o erro número um de quem está começando: criar a coluna telefone como INT.

Se você usar INT	Use VARCHAR
O zero da frente some: o DDD 047 vira 47	Guarda exatamente o que foi digitado
Não cabe parêntese nem traço: “(47) 99999-1111” não é um número	Aceita parêntese, traço, ponto e espaço
E para quê? Você nunca vai somar dois telefones	Vale também para CEP, CPF, RG e código de produto

Regra para levar para a vida. Número que você não soma não é número — é texto. Telefone, CEP, CPF, RG, número de cartão e código de barras são todos VARCHAR.

2.6 O campo pode ficar vazio? NOT NULL e NULL

Ao criar a tabela, você decide quais colunas são obrigatórias. O banco usa a palavra NULL para dizer “este campo está vazio”.

- **`NOT NULL`** quer dizer: esta coluna é obrigatória. Se alguém tentar cadastrar sem preencher, o banco recusa. Use em nome e telefone — o que não pode faltar.
- **`NULL`** quer dizer: esta coluna pode ficar vazia. O cadastro é aceito mesmo sem ela. Use em email e observação — o que nem todo mundo tem.

NULL não é zero e não é texto vazio. NULL significa “não sabemos” ou “não se aplica”. É por isso que, para procurar quem está sem e-mail, se escreve email IS NULL, e não email = NULL.

2.7 O id e a chave primária

Imagine duas pessoas com o mesmo nome na sua lista:

id	nome	telefone
1	Maria Silva	47999991111
2	João Pereira	47999982222
7	Maria Silva	47988887777

Como você diz ao banco qual das duas Marias quer alterar? Pelo nome, não dá: as duas têm o mesmo. Existem duas Marias Silva, mas só existe um id 1 e só existe um id 7. O id resolve a confusão.

> **A definição.** Chave primária é a coluna que identifica cada linha da tabela de forma única. Ela nunca se repete, nunca fica vazia e nunca muda.

E o melhor: você não precisa inventar o número. O banco conta para você.

```
CREATE TABLE contatos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);
```

- **`AUTO_INCREMENT`** quer dizer “vai contando sozinho”. O primeiro contato recebe o id 1, o segundo recebe 2, e assim por diante.
- **`PRIMARY KEY`** quer dizer “esta é a coluna que identifica a linha”.

Como o banco conta sozinho, você nunca vai digitar o id ao cadastrar um contato.

3. Criando o primeiro banco

3.1 O que você precisa instalado

- **XAMPP.** Um pacote único que instala o MySQL e mais algumas ferramentas de uma vez só. É gratuito e existe para Windows, Mac e Linux. WAMP e Laragon são alternativas que servem igual.
- **phpMyAdmin.** Vem junto com o XAMPP. É a tela onde vamos digitar os comandos SQL, direto no navegador.
- **MySQL Workbench.** Alternativa ao phpMyAdmin, instalada como programa separado. Use se preferir: os comandos são exatamente os mesmos.

Para ligar o MySQL: abra o painel de controle do XAMPP, clique em Start no MySQL (ele precisa ficar verde), clique em Start também no Apache, e depois acesse localhost/phpmyadmin no navegador.

Se der problema	Provavelmente é	O que fazer
O Apache não inicia	A porta 80 já está sendo usada por outro programa	Trocar a porta do Apache para 8080 nas configurações
O MySQL não inicia	Já existe outro MySQL instalado na máquina	Parar o serviço antigo pelo Gerenciador de Tarefas

3.2 Onde escrever os comandos

No phpMyAdmin, clique na aba SQL do menu de cima, digite o comando na caixa de texto e clique em Executar. No MySQL Workbench, digite na janela de query e clique no raio (Execute) ou pressione Ctrl+Enter.

Os comandos são exatamente os mesmos nos dois programas. O que muda é só onde fica o botão de executar.

3.3 Cinco regras para escrever SQL

- **Termine com ponto e vírgula.** Todo comando termina com ; — é assim que o banco sabe que a frase acabou.
- **MAIÚSCULA é só costume.** SELECT e select funcionam igual. Escrevemos os comandos em maiúscula apenas para destacá-los.
- **Texto vai entre aspas simples.** 'Maria Silva' com aspas. Números não levam aspas: 42 é 42.
- **Nomes sem acento e sem espaço.** A coluna chama data_cadastro, não “data cadastro” nem “data-cadastro”. Use o sublinhado para separar palavras.
- **Leia a mensagem de erro.** Ela quase sempre diz exatamente qual palavra está errada e em que posição.

3.4 Comando 1: criar o banco

```
CREATE DATABASE agenda;
```

CREATE quer dizer criar. DATABASE quer dizer banco de dados. E agenda é o nome que escolhemos. Traduzindo: “crie um banco de dados chamado agenda”. Depois de executar, o banco agenda aparece na lista do lado esquerdo da tela.

A versão completa, que é a que vamos usar de verdade, inclui o conjunto de caracteres:

```
CREATE DATABASE agenda CHARACTER SET utf8mb4;
```

Por que utf8mb4. É o que garante que acento e cedilha sejam guardados corretamente. Sem isso, “João” pode aparecer na tela como “JoÃ£o”.

3.5 Comando 2: entrar no banco

```
USE agenda;
```

Traduzindo: “a partir de agora, todos os comandos são neste banco”. Pense assim: o CREATE DATABASE criou uma pasta, e o USE abre essa pasta. Tudo o que você fizer daqui em diante acontece dentro dela.

No phpMyAdmin você também pode simplesmente clicar no nome do banco na lista da esquerda — dá no mesmo.

Se você esquecer o USE o MySQL responde “No database selected”. É um dos erros mais comuns da primeira aula, e a solução é só rodar o USE.

3.6 Comando 3: criar a tabela

A estrutura do comando é sempre a mesma:

```
CREATE TABLE nome_da_tabela (
    nome_da_coluna   TIPO,
    nome_da_coluna   TIPO
);
```

Repare em quatro detalhes: os parênteses abrem depois do nome da tabela e fecham no final; cada coluna fica em uma linha, com o nome primeiro e o tipo depois; as colunas são separadas por vírgula, e a última NÃO tem vírgula; e o ponto e vírgula fecha o comando.

Agora com conteúdo de verdade, a nossa tabela de contatos:

```
CREATE TABLE contatos (
    id       INT AUTO_INCREMENT PRIMARY KEY,
    nome     VARCHAR(100) NOT NULL,
    telefone VARCHAR(20)  NOT NULL,
    email    VARCHAR(150)
);
```

- **`id`** — número que o banco cria sozinho e que identifica a linha.
- **`nome`** — texto de até 100 caracteres, obrigatório.
- **`telefone`** — texto de até 20 caracteres, obrigatório. Texto, não número.
- **`email`** — texto de até 150 caracteres. Sem NOT NULL, então pode ficar vazio.

Repare que só as três primeiras colunas têm vírgula no final. A última, email, não tem — depois dela vem o parêntese que fecha.

3.7 Conferindo se deu certo

Ganhe o hábito de conferir. Depois de criar qualquer coisa, olhe se ela existe.

```
SHOW TABLES;

DESCRIBE contatos;
```

SHOW TABLES lista todas as tabelas que existem no banco em que você está — deve aparecer contatos. E DESCRIBE mostra as colunas da tabela, com o tipo de cada uma e se ela aceita ficar vazia. O resultado deve ser parecido com este:

Field	Type	Null	Key	Extra
id	int	NO	PRI	auto_increment
nome	varchar(100)	NO		
telefone	varchar(20)	NO		
email	varchar(150)	YES		

4. Os quatro comandos

4.1 O CRUD

São quatro coisas que se faz com um dado guardado, e a sigla CRUD junta as quatro. Ela aparece o tempo todo no mercado de trabalho: “esse sistema é só um CRUD” é uma frase que você vai ouvir.

Letra	Significa	Comando	Em português
C	Create — criar	`INSERT`	“Cadastre este contato novo.”
R	Read — ler	`SELECT`	“Me mostre os contatos.”
U	Update — atualizar	`UPDATE`	“Troque o telefone da Maria.”
D	Delete — apagar	`DELETE`	“Remova este contato.”

Praticamente todo sistema que você já usou faz essas quatro coisas. Aprendendo os quatro comandos, você já consegue construir um sistema simples.

4.2 INSERT: guardar um contato

```
INSERT INTO contatos (nome, telefone, email)
VALUES ('Maria Silva', '47999991111', 'maria@email.com');
```

- **`INSERT INTO contatos`** — “guarde na tabela contatos”.
- **`(nome, telefone, email)`** — quais colunas você vai preencher.
- **`VALUES (...)`** — os valores, na mesma ordem das colunas listadas acima.

Repare que o id não aparece: o banco cria sozinho, porque a coluna é AUTO_INCREMENT.

> **Atenção à ordem.** A ordem dos valores tem que bater com a ordem das colunas. Se trocar, o telefone vai parar no campo do e-mail — e o banco aceita, porque os dois são texto.

Para cadastrar vários de uma vez, cada contato vai entre parênteses, separado por vírgula. Quem não tem e-mail recebe NULL, sem aspas:

```
INSERT INTO contatos (nome, telefone, email) VALUES
    ('Joao Pereira', '47999982222', 'joao@email.com'),
    ('Ana Souza',    '47999973333', 'ana@email.com'),
    ('Diego Alves',  '47988776655', NULL);
```

4.3 SELECT: ver o que está guardado

```
SELECT * FROM contatos;
```

SELECT quer dizer “mostre”. O asterisco quer dizer “todas as colunas”. E FROM contatos quer dizer “da tabela contatos”. Traduzindo: “mostre todas as colunas de todas as linhas da tabela contatos”.

Se você quiser só algumas colunas, é só listar quais:

```
SELECT nome, telefone FROM contatos;
```

4.4 WHERE: filtrar as linhas

Sem WHERE, o SELECT traz a tabela inteira. Com WHERE, traz só o que atende à condição. A palavra quer dizer “nos casos em que”.

```
SELECT * FROM contatos WHERE id = 3;
SELECT * FROM contatos WHERE nome = 'Ana Souza';
```

Repare nas aspas: texto vai entre aspas simples, número não vai.

Sinal	Quer dizer	Exemplo
`=`	igual a	`WHERE id = 3`
`<>`	diferente de	`WHERE nome <> 'Ana Souza'`
`>`	maior que	`WHERE id > 5`
`<`	menor que	`WHERE id < 10`
`>=`	maior ou igual a	`WHERE id >= 5`
`<=`	menor ou igual a	`WHERE id <= 10`

Para comparar, o sinal de igual é um só. Escreve-se WHERE id = 3. Dois sinais (==) dão erro no SQL, diferente de outras linguagens de programação.

4.5 LIKE: buscar por parte do texto

Quando você não sabe o nome inteiro, usa LIKE com o sinal de porcentagem. O % quer dizer “qualquer coisa aqui”. É assim que se implementa o campo de busca de um sistema.

```
-- começa com Mar
SELECT * FROM contatos WHERE nome LIKE 'Mar%';

-- termina com Silva
SELECT * FROM contatos WHERE nome LIKE '%Silva';

-- contem Souza em qualquer lugar
SELECT * FROM contatos WHERE nome LIKE '%Souza%';
```

E para procurar quem está sem e-mail cadastrado, use IS NULL — nunca “= NULL”:

```
SELECT * FROM contatos WHERE email IS NULL;
SELECT * FROM contatos WHERE email IS NOT NULL;
```

4.6 ORDER BY: colocar em ordem

```
SELECT * FROM contatos ORDER BY nome;        -- de A a Z
SELECT * FROM contatos ORDER BY nome DESC;   -- de Z a A
SELECT * FROM contatos ORDER BY id DESC;     -- do mais recente
```

ASC quer dizer crescente e é o padrão, então não precisa escrever. DESC quer dizer decrescente. Você também pode combinar filtro e ordem no mesmo comando: primeiro o WHERE, depois o ORDER BY.

```
SELECT nome, telefone
FROM   contatos
WHERE  email IS NOT NULL
ORDER  BY nome;
```

Sem ORDER BY o banco não garante em que ordem as linhas aparecem. Se a ordem importa para você, escreva o ORDER BY.

4.7 UPDATE: corrigir uma informação

```
UPDATE contatos
SET    telefone = '47988887777'
WHERE  id = 3;
```

- **`UPDATE contatos`** — “altere a tabela contatos”.
- **`SET telefone = '...'`** — “coloque este valor nesta coluna”.
- **`WHERE id = 3`** — “mas só na linha de id 3”.

Para alterar duas colunas de uma vez, separe com vírgula depois do SET: SET nome = 'Ana Souza Lima', telefone = '4799990000'.

4.8 O comando mais perigoso do SQL

```
UPDATE contatos SET nome = 'X';
```

Sem o WHERE, esse comando muda o nome de TODOS os contatos da tabela para “X”. Não pergunta nada. Não avisa. Não tem como desfazer.

Três hábitos evitam isso para sempre. Primeiro: escreva o SELECT com o WHERE, veja quais linhas aparecem, e só então troque SELECT * por UPDATE. Segundo: digite o WHERE antes do SET, assim, se executar sem querer no meio do caminho, o comando está incompleto e dá erro. Terceiro: antes de executar, releia e pergunte a si mesmo “tem WHERE aí?”.

4.9 DELETE: apagar um contato

```
-- 1o: confira QUAL linha vai sumir
SELECT * FROM contatos WHERE id = 5;

-- 2o: so entao apague
DELETE FROM contatos WHERE id = 5;
```

DELETE FROM contatos; sem WHERE apaga a tabela inteira, linha por linha. Vale o mesmo cuidado do UPDATE.

Um detalhe que confunde: se você apagar o contato de id 5, esse número não volta a ser usado. O próximo contato cadastrado recebe o id seguinte ao maior que já existiu. Isso é de propósito — reaproveitar identificador é receita para associar informação à pessoa errada.

4.10 A regra dos dois passos

Vale para UPDATE e para DELETE, sempre, sem exceção.

- **Passo 1.** Escreva o SELECT com o WHERE que você pretende usar. Execute e olhe o resultado.
- **Passo 2.** Confira: apareceram exatamente as linhas que você queria mexer? Nem mais, nem menos?

- **Passo 3.** Agora sim: troque o SELECT * por UPDATE ... SET ou por DELETE, mantendo o mesmo WHERE.

Por que isso importa tanto. Dez segundos conferindo o SELECT evitam uma tarde inteira tentando recuperar dados. Este é o hábito que separa quem já quebrou um banco de quem ainda vai quebrar.

5. Quando der errado

Errar em SQL é normal e não quebra nada: o banco recusa o comando e explica o motivo. A mensagem de erro é sua aliada, não uma bronca. Esta tabela cobre praticamente todos os erros que aparecem na primeira aula.

A mensagem diz	O que significa	O que fazer
No database selected	Você não disse em qual banco quer trabalhar	Rode USE agenda; antes do comando
Unknown database 'agenda'	O banco não foi criado, ou o nome está diferente	Rode o CREATE DATABASE e confira a grafia
Table 'agenda.contatos' doesn't exist	A tabela não foi criada, ou está em outro banco	Rode o CREATE TABLE depois do USE agenda
Unknown column 'nomes' in 'field list'	O nome da coluna está errado no comando	Rode DESCRIBE contatos e confira o nome exato
Field 'nome' doesn't have a default value	Faltou preencher uma coluna obrigatória	Inclua a coluna que falta no INSERT
You have an error in your SQL syntax	Erro de digitação: vírgula, aspa, parêntese ou ponto e vírgula	Leia o trecho citado; o erro está ali ou logo antes
Data too long for column 'telefone'	O texto é maior do que o tamanho da coluna	Encurte o valor ou aumente o VARCHAR
Acentos aparecem como “JoÃ£o”	O banco não foi criado com utf8mb4	Recrie o banco com CHARACTER SET utf8mb4

6. Exercícios

Faça no computador, um de cada vez. O gabarito está logo depois — tente antes de olhar.

- E1. Mostre todos os contatos, com todas as colunas.
- E2. Mostre apenas o nome e o e-mail de todos os contatos.
- E3. Mostre o contato de id 4.
- E4. Mostre os contatos cujo nome começa com a letra A.
- E5. Mostre todos os contatos em ordem alfabética de nome.
- E6. Mostre os contatos que estão sem e-mail cadastrado.
- E7. Cadastre um contato chamado 'Gabriela Nunes', telefone '47993334444', sem e-mail.
- E8. Troque o telefone da Gabriela para '47991112222'. Lembre da regra dos dois passos.
- E9. Apague a Gabriela da tabela. Lembre da regra dos dois passos.
- E10. Mostre os contatos com id maior que 3, em ordem alfabética de nome.

Gabarito

```
-- E1
SELECT * FROM contatos;
-- E2
SELECT nome, email FROM contatos;
-- E3
SELECT * FROM contatos WHERE id = 4;
-- E4
SELECT * FROM contatos WHERE nome LIKE 'A%';
-- E5
SELECT * FROM contatos ORDER BY nome;
-- E6
SELECT * FROM contatos WHERE email IS NULL;
-- E7
INSERT INTO contatos (nome, telefone, email)
VALUES ('Gabriela Nunes', '47993334444', NULL);
-- E8
SELECT * FROM contatos WHERE nome = 'Gabriela Nunes';
UPDATE contatos SET telefone = '47991112222'
WHERE nome = 'Gabriela Nunes';
-- E9
SELECT * FROM contatos WHERE nome = 'Gabriela Nunes';
DELETE FROM contatos WHERE nome = 'Gabriela Nunes';
-- E10
SELECT * FROM contatos WHERE id > 3 ORDER BY nome;
```

Escreveu diferente e funcionou? Também está certo. Em SQL quase sempre existe mais de um jeito de chegar ao mesmo resultado.

Desafio final: o banco da loja

Faça sozinho, do zero, usando tudo o que você aprendeu.

- Crie um banco chamado loja, com CHARACTER SET utf8mb4.
- Crie uma tabela produtos com as colunas: id, nome, preco, quantidade e codigo_barras.
- Cadastre cinco produtos.
- Mostre os produtos em ordem alfabética de nome.
- Mostre só os produtos com quantidade menor que 10.

- Altere o preço de um produto e apague outro, os dois com WHERE e com SELECT antes.

Duas pegadinhas propositais. Que tipo usar em preco? Ele tem centavos. E em codigo_barras? Ele tem zero na frente, e você nunca vai somar dois códigos de barras. Pense antes de responder.

Gabarito da criação da tabela — confira só depois de tentar:

```
CREATE DATABASE loja CHARACTER SET utf8mb4;
USE loja;

CREATE TABLE produtos (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    nome          VARCHAR(100)  NOT NULL,
    preco         DECIMAL(10,2) NOT NULL,
    quantidade    INT           NOT NULL,
    codigo_barras VARCHAR(20)
);
```

O preco é DECIMAL porque tem centavos, e um número inteiro não guardaria a parte depois da vírgula. O codigo_barras é VARCHAR porque tem zero na frente — se fosse INT, o zero sumiria — e porque ninguém soma dois códigos de barras.

Colinha de comandos

Todos os comandos da aula em um lugar só. Se você for imprimir uma página deste material, imprima esta.

Criar

```
CREATE DATABASE agenda CHARACTER SET utf8mb4;
USE agenda;

CREATE TABLE contatos (
    id       INT AUTO_INCREMENT PRIMARY KEY,
    nome     VARCHAR(100) NOT NULL,
    telefone VARCHAR(20)  NOT NULL,
    email    VARCHAR(150)
);

SHOW TABLES;
DESCRIBE contatos;
```

Guardar

```
INSERT INTO contatos (nome, telefone, email)
VALUES ('Maria Silva', '47999991111', 'maria@email.com');
```

Consultar

```
SELECT * FROM contatos;
SELECT nome, telefone FROM contatos;
SELECT * FROM contatos WHERE id = 3;
SELECT * FROM contatos WHERE nome LIKE 'Mar%';
SELECT * FROM contatos WHERE email IS NULL;
SELECT * FROM contatos ORDER BY nome;
```

Alterar e apagar

```
UPDATE contatos SET nome = 'Ana' WHERE id = 3;

DELETE FROM contatos WHERE id = 3;
```

> **Lembre sempre.** UPDATE e DELETE sem WHERE pegam a tabela inteira. Faça sempre um SELECT com o mesmo WHERE antes de executar.

Material produzido para a disciplina de Análise e Desenvolvimento de Sistemas do UniSENAI Santa Catarina. Prof. Me. Janderson Barbosa da Silva.